



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения

**ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ №7
по дисциплине
«Технологии виртуализации клиент-серверных приложений»**

Выполнил студент группы ИКБО-16-22

Грибков А.С.

Принял преподаватель кафедры ИиППО

Волков М.Ю.

Практические работы выполнены

«__»_____2025 г.

«Зачтено»

«__»_____2025 г.

Москва
2025

СОДЕРЖАНИЕ

1 Цель работы	3
2 Ход работы	4
2.1 Запуск сервера метрик	4
2.2 Проверка работы сервера метрик.....	4
2.3 Создание пространства имён.....	5
3 Управление ресурсами памяти.....	6
3.1 Создание Pod с ограничениями памяти.....	6
3.2 Создание Pod, превышающего лимит памяти	8
3.3 Создание Pod с недостижимым запросом памяти.....	10
4 Управление ресурсами CPU	12
4.1 Создание Pod с ограничениями CPU	12
4.2 Создание Pod с недостижимым запросом CPU	13
5 Вывод	15
6 Ответы на вопросы	16
7 Список использованной литературы	19

1 ЦЕЛЬ РАБОТЫ

Изучить механизмы управления ресурсами (CPU и память) в Kubernetes через настройку requests и limits для контейнеров.

Задачи работы:

1. Запустить сервер метрик в minikube
2. Создать пространство имён для изоляции ресурсов
3. Настроить ограничения памяти для контейнеров
4. Изучить поведение контейнеров при превышении лимитов памяти
5. Настроить ограничения CPU для контейнеров
6. Понять механизмы планирования подов при недостаточных ресурсах
7. Изучить QoS-классы подов в Kubernetes

2 ХОД РАБОТЫ

2.1 Запуск сервера метрик

Metrics Server необходим для сбора метрик использования ресурсов (CPU и памяти) с узлов и подов кластера. Для включения сервера метрик в Minikube используется команда:

Листинг 2.1 — Команда включения metrics-server

```
minikube addons enable metrics-server
```

```
alexgribkov@i113971732 practical-work-7 % minikube addons enable metrics-server
💡 metrics-server is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
▪ Using image registry.k8s.io/metrics-server/metrics-server:v0.8.0
🌟 The 'metrics-server' addon is enabled
```

Рисунок 2.1 — Включение аддона metrics-server в Minikube

2.2 Проверка работы сервера метрик

После включения metrics-server необходимо проверить доступность API метрик. Выполняется команда для просмотра списка API-сервисов:

Листинг 2.2 — Команда проверки API-сервисов

```
kubectl get apiservices
```

В выводе команды должна присутствовать строка v1beta1.metrics.k8s.io, что подтверждает доступность API ресурсов метрики.

```
alexgribkov@i113971732 practical-work-7 % kubectl get apiservices v1
beta1.metrics.k8s.io
NAME                                SERVICE                                AVAILABLE  AGE
v1beta1.metrics.k8s.io             kube-system/metrics-server            True      3d
```

Рисунок 2.2 — Проверка доступности API метрик

2.3 Создание пространства имён

Создается пространство имён для изоляции ресурсов практической работы от остальных ресурсов кластера:

Листинг 2.3 — Команда создания namespace

```
kubectl create namespace gribkov-as-ikbo-16-22
```

Использование отдельного namespace обеспечивает логическую изоляцию ресурсов и упрощает управление и очистку после завершения работы.

```
alexgribkov@i113971732 practical-work-7 % kubectl create namespace g
ribkov-as-ikbo-16-22
namespace/gribkov-as-ikbo-16-22 created
```

Рисунок 2.3 — Создание пространства имён

3 УПРАВЛЕНИЕ РЕСУРСАМИ ПАМЯТИ

3.1 Создание Pod с ограничениями памяти

Создается Pod с корректными ограничениями памяти. Для контейнера устанавливается запрос памяти (request) в 100 MiB и лимит (limit) в 200 MiB. Контейнер попытается использовать 150 MiB памяти, что находится в пределах установленного лимита.

Листинг 3.1 — Конфигурационный файл memory-request-limit.yaml

```
apiVersion: v1 kind: Pod metadata: name: memory-demo namespace:
gribkov-as-ikbo-16-22 spec: containers: - name: memory-demo-ctr
image: polinux/stress resources: limits: memory: "200Mi" requests:
memory: "100Mi" command: ["stress"] args: ["-vm", "1", "-vm-
bytes", "150M", "-vm-hang", "1"]
```

Раздел args содержит аргументы для контейнера. Аргументы `-vm-bytes 150M` указывают контейнеру попытаться занять 150 MiB памяти.

```
alexgribkov@i113971732 practical-work-7 % kubectl apply -f memory-re
quest-limit.yaml
pod/memory-demo created
alexgribkov@i113971732 practical-work-7 % kubectl get pod memory-dem
o --namespace=gribkov-as-ikbo-16-22
```

NAME	READY	STATUS	RESTARTS	AGE
memory-demo	1/1	Running	0	4s

Рисунок 3.1 — Создание Pod с ограничениями памяти

После создания пода проверяется его статус:

Листинг 3.2 — Команда проверки статуса Pod

```
kubectl get pod memory-demo -namespace=gribkov-as-ikbo-16-22
alexgribkov@i113971732 practical-work-7 % kubectl get pod memory-dem
o --namespace=gribkov-as-ikbo-16-22
```

NAME	READY	STATUS	RESTARTS	AGE
memory-demo	1/1	Running	0	43s

Рисунок 3.2 — Статус Pod memory-demo

Для получения подробной информации о ресурсах пода используется команда:

Листинг 3.3 — Команда просмотра детальной информации

```
kubectl get pod memory-demo -output=yaml -namespace=gribkov-as-ikbo-16-22
```

В выводе должно быть видно, что для контейнера зарезервировано 100 MiB памяти (requests) и установлено ограничение в 200 MiB (limits).

```
alexgribkov@i113971732 practical-work-7 % kubectl get pod memory-demo --output=yaml --namespace=gribkov-as-ikbo-16-22 | grep -A 10 "resources:"

resources:
  limits:
    memory: 200Mi
  requests:
    memory: 100Mi
  terminationMessagePath: /dev/termination-log
  terminationMessagePolicy: File
  volumeMounts:
  - mountPath: /var/run/secrets/kubernetes.io/serviceaccount
    name: kube-api-access-jzrsc
    readOnly: true
--
resources:
  limits:
    memory: 200Mi
  requests:
    memory: 100Mi
  restartCount: 1
  started: true
  state:
    running:
      startedAt: "2025-11-23T20:20:20Z"
  volumeMounts:
```

Рисунок 3.3 — Детальная информация о ресурсах Pod

Для получения метрик использования ресурсов подом выполняется команда:

Листинг 3.4 — Команда получения метрик Pod

```
kubectl top pod memory-demo -namespace=gribkov-as-ikbo-16-22
```

Вывод показывает фактическое использование памяти подом, которое должно быть близко к 150 MiB.

```
alexgribkov@i113971732 practical-work-7 % kubectl top pod memory-demo --namespace=gribkov-as-ikbo-16-22
```

NAME	CPU(cores)	MEMORY(bytes)
memory-demo	100m	165Mi

Рисунок 3.4 — Метрики использования памяти Pod memory-demo

3.2 Создание Pod, превышающего лимит памяти

Создается Pod, который попытается использовать больше памяти, чем установленный лимит. Контейнер имеет запрос в 50 MiB и лимит в 100 MiB, но попытается занять 250 MiB памяти.

Листинг 3.5 — Конфигурационный файл memory-request-limit-2.yaml

```
apiVersion: v1 kind: Pod metadata: name: memory-demo-2 namespace: gribkov-as-ikbo-16-22 spec: containers: - name: memory-demo-2-ctr image: polinux/stress resources: requests: memory: "50Mi" limits: memory: "100Mi" command: ["stress"] args: ["-vm", "1", "-vm-bytes", "250M", "-vm-hang", "1"]
```

```
alexgribkov@i113971732 practical-work-7 % kubectl apply -f memory-request-limit-2.yaml
```

```
pod/memory-demo-2 created
```

Рисунок 3.5 — Создание Pod, превышающего лимит памяти

После создания пода необходимо наблюдать за его статусом. Контейнер будет убит по причине превышения лимита памяти (OOMKilled) и перезапущен kubelet.

Листинг 3.6 — Команда проверки статуса Pod

```
kubectl get pod memory-demo-2 -namespace=gribkov-as-ikbo-16-22
```

Статус пода будет показывать CrashLoopBackOff, что означает циклическое завершение и перезапуск контейнера.

```
alexgribkov@i113971732 practical-work-7 % kubectl get pod memory-demo-2 --namespace=gribkov-as-ikbo-16-22
```

NAME	READY	STATUS	RESTARTS	AGE
memory-demo-2	0/1	CrashLoopBackOff	1 (14s ago)	19s

Рисунок 3.6 — Статус Pod в состоянии CrashLoopBackOff

Для просмотра подробной информации об истории пода используется:

Листинг 3.7 — Команда просмотра детальной информации

```
kubectl get pod memory-demo-2 -output=yaml -namespace=gribkov-as-ikbo-16-22
```

В секции `lastState` вывода будет видно, что контейнер был завершен с причиной `OOMKilled` и кодом выхода `137`.

```
alexgribkov@i113971732 practical-work-7 % kubectl get pod memory-demo-2 --output=yaml --namespace=gribkov-as-ikbo-16-22 | grep -A 10 "lastState:"
```

```
    lastState:
      terminated:
        containerID: docker://eee4dcb5fcda774d91e95248bbf763e4b61b4a84fcb76993646eecead61cc957
        exitCode: 1
        finishedAt: "2025-11-23T20:22:06Z"
        reason: OOMKilled
        startedAt: "2025-11-23T20:22:06Z"
    name: memory-demo-2-ctr
    ready: false
    resources:
      limits:
```

Рисунок 3.7 — Детальная информация о завершении контейнера с OOMKilled

Для получения событий, связанных с нодой, выполняется команда:

Листинг 3.8 — Команда описания нод

```
kubectl describe nodes
```

В выводе должно содержаться предупреждение о том, что контейнер был убит по причине нехватки памяти (`OOMKilling Memory`).

```
alexgribkov@i113971732 practical-work-7 % kubectl describe pod memory-demo-2 --namespace=gribkov-as-ikbo-16-22 | grep -A 5 "Events:"
```

Events:

Type	Reason	Age	From	Message
Normal	Scheduled	90s	default-scheduler	Successfully assigned gribkov-as-ikbo-16-22/memory-demo-2 to minikube
Normal	Pulled	89s	kubelet	Successfully pulled image "polinux/stress" in 1.365s (1.365s including waiting). Image size: 9744175 bytes.
Normal	Pulled	87s	kubelet	Successfully pulled image "polinux/stress" in 1.284s (1.284s including waiting). Image size: 9744175 bytes.

Рисунок 3.8 — Информация о событиях OOMKilling на нодe

3.3 Создание Pod с недостижимым запросом памяти

Создается Pod с запросом памяти, превышающим емкость любой ноды в кластере. Контейнер запрашивает 1000 GiB памяти, что гарантированно превышает доступные ресурсы.

Листинг 3.9 — Конфигурационный файл *memory-request-limit-3.yaml*

```
apiVersion: v1 kind: Pod metadata: name: memory-demo-3 namespace: gribkov-as-ikbo-16-22 spec: containers: - name: memory-demo-3-ctr image: polinux/stress resources: limits: memory: "1000Gi" requests: memory: "1000Gi" command: ["stress"] args: ["-vm", "1", "-vm-bytes", "150M", "-vm-hang", "1"]
```

```
alexgribkov@i113971732 practical-work-7 % kubectl apply -f memory-request-limit-3.yaml
pod/memory-demo-3 created
```

Рисунок 3.9 — Создание Pod с недостижимым запросом памяти

После создания пода проверяется его статус:

Листинг 3.10 — Команда проверки статуса Pod

```
kubectl get pod memory-demo-3 -namespace=gribkov-as-ikbo-16-22
```

Pod будет иметь статус Pending, что означает, что он не может быть запланирован ни на одной нодe из-за нехватки ресурсов.

```
alexgribkov@i113971732 practical-work-7 % kubectl get pod memory-demo-3 --namespace=gribkov-as-ikbo-16-22
```

NAME	READY	STATUS	RESTARTS	AGE
memory-demo-3	0/1	Pending	0	21s

Рисунок 3.10 — Статус Pod в состоянии Pending

Для просмотра причины невозможности планирования выполняется:

Листинг 3.11 — Команда описания Pod

```
kubectl describe pod memory-demo-3 -namespace=gribkov-as-ikbo-16-22
```

В секции Events будет указано сообщение: FailedScheduling - No nodes are available that match all of the following predicates: Insufficient memory.

```
alexgribkov@i113971732 practical-work-7 % kubectl describe pod memory-demo-3 --namespace=gribkov-as-ikbo-16-22 | grep -A 10 "Events:"
```

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Warning	FailedScheduling	40s	default-scheduler	0/1 nodes are available: 1 Insufficient memory. no new claims to deallocate, preemption: 0/1 nodes are available: 1 Preemption is not helpful for scheduling.

Рисунок 3.11 — События Pod с ошибкой планирования

4 УПРАВЛЕНИЕ РЕСУРСАМИ CPU

4.1 Создание Pod с ограничениями CPU

Создается Pod с ограничениями CPU. Контейнер запрашивает 0.5 CPU (500 милли-CPU) и имеет лимит в 1 CPU. Контейнер попытается использовать 2 CPU, но будет ограничен установленным лимитом.

Листинг 4.1 — Конфигурационный файл `cpu-request-limit.yaml`

```
apiVersion: v1 kind: Pod metadata: name: cpu-demo namespace:
gribkov-as-ikbo-16-22 spec: containers: - name: cpu-demo-ctr
image: polinux/stress resources: limits: cpu: "1" requests: cpu:
"0.5" command: ["stress"] args: - -cpu - "2"
```

Аргумент `-cpu 2` указывает контейнеру попытаться использовать 2 CPU, однако контейнер будет ограничен лимитом в 1 CPU.

```
alexgribkov@i113971732 practical-work-7 % kubectl apply -f cpu-reque
st-limit.yaml

pod/cpu-demo created
```

Рисунок 4.1 — Создание Pod с ограничениями CPU

После создания пода проверяется его статус:

Листинг 4.2 — Команда проверки статуса Pod

```
kubectl get pod cpu-demo -namespace=gribkov-as-ikbo-16-22

alexgribkov@i113971732 practical-work-7 % kubectl get pod cpu-demo -
-namespace=gribkov-as-ikbo-16-22
```

NAME	READY	STATUS	RESTARTS	AGE
cpu-demo	1/1	Running	0	30s

Рисунок 4.2 — Статус Pod `cpu-demo`

Для просмотра детальной информации о ресурсах пода используется:

Листинг 4.3 — Команда просмотра детальной информации

```
kubectl get pod cpu-demo -output=yaml -namespace=gribkov-as-ikbo-
16-22
```

В выводе видно, что pod имеет запрос в 500 милли-CPU (0.5 CPU) и ограничение в 1 CPU.

```
alexgribkov@i113971732 practical-work-7 % kubectl get pod cpu-demo -
--namespace=gribkov-as-ikbo-16-22

NAME          READY   STATUS    RESTARTS   AGE
cpu-demo      1/1     Running   0           30s
alexgribkov@i113971732 practical-work-7 % kubectl describe pod cpu-d
emo --namespace=gribkov-as-ikbo-16-22 | grep -A 5 "Limits:"

Limits:
  cpu: 1
Requests:
  cpu:      500m
Environment:  <none>
Mounts:
```

Рисунок 4.3 — Детальная информация о ресурсах CPU для Pod

Для получения метрик использования CPU подом выполняется:

Листинг 4.4 — Команда получения метрик Pod

```
kubectl top pod cpu-demo --namespace=gribkov-as-ikbo-16-22
```

В выводе показано использование CPU подом. Значение будет близко к 974 милли-CPU, что немного меньше установленного лимита в 1 CPU.

```
alexgribkov@i113971732 practical-work-7 % kubectl top pod cpu-demo -
--namespace=gribkov-as-ikbo-16-22

NAME          CPU(cores)   MEMORY(bytes)
cpu-demo      982m         5Mi
```

Рисунок 4.4 — Метрики использования CPU для Pod cpu-demo

4.2 Создание Pod с недостижимым запросом CPU

Создается Pod с запросом CPU, превышающим мощности любой ноды в кластере. Контейнер запрашивает 100 CPU, что значительно превышает доступные ресурсы.

Листинг 4.5 — Конфигурационный файл cpu-request-limit-2.yaml

```
apiVersion: v1 kind: Pod metadata: name: cpu-demo-2 namespace:
gribkov-as-ikbo-16-22 spec: containers: - name: cpu-demo-ctr-2
image: polinux/stress resources: limits: cpu: "100" requests: cpu:
"100" command: ["stress"] args: - -cpu - "2"

alexgribkov@i113971732 practical-work-7 % kubectl apply -f cpu-reque
st-limit-2.yaml

pod/cpu-demo-2 created
```

Рисунок 4.5 — Создание Pod с недостижимым запросом CPU

После создания пода проверяется его статус:

Листинг 4.6 — Команда проверки статуса Pod

```
kubectl get pod cpu-demo-2 -namespace=gribkov-as-ikbo-16-22
```

Pod имеет статус Pending, так как не может быть запланирован ни на одной ноде из-за недостаточных ресурсов CPU.

```
alexgribkov@i113971732 practical-work-7 % kubectl get pod cpu-demo-2
--namespace=gribkov-as-ikbo-16-22
```

NAME	READY	STATUS	RESTARTS	AGE
cpu-demo-2	0/1	Pending	0	16s

Рисунок 4.6 — Статус Pod cpu-demo-2 в состоянии Pending

Для просмотра причины невозможности планирования используется:

Листинг 4.7 — Команда описания Pod

```
kubectl describe pod cpu-demo-2 -namespace=gribkov-as-ikbo-16-22
```

В секции Events будет сообщение: FailedScheduling - No nodes are available that match all of the following predicates: Insufficient cpu.

```
alexgribkov@i113971732 practical-work-7 % kubectl describe pod cpu-d
emo-2 --namespace=gribkov-as-ikbo-16-22 | grep -A 10 "Events:"
```

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Warning	FailedScheduling	41s	default-scheduler	0/1 nodes are available: 1 Insufficient cpu. no new claims to deallocate, preempti on: 0/1 nodes are available: 1 Preemption is not helpful for schedul ing.

Рисунок 4.7 — События Pod с ошибкой планирования из-за нехватки CPU

5 ВЫВОД

В результате выполнения практической работы были изучены механизмы управления ресурсами в Kubernetes. Достигнуты следующие результаты:

1. Запущен сервер метрик в minikube для сбора информации об использовании ресурсов
2. Создано отдельное пространство имён для изоляции ресурсов практической работы
3. Изучены механизмы работы requests и limits для памяти:
 - Pod с корректными ограничениями успешно работает в пределах выделенных ресурсов
 - При превышении лимита памяти контейнер убивается kubelet с ошибкой OOMKilled
 - Pod с недостижимым запросом памяти остается в статусе Pending
4. Изучены механизмы работы requests и limits для CPU:
 - CPU ограничивается через throttling, контейнер не завершается при попытке превысить лимит
 - Pod с недостижимым запросом CPU не может быть запланирован на ноде
5. Понято назначение QoS-классов в Kubernetes:
 - Guaranteed: requests равны limits для всех ресурсов
 - Burstable: requests меньше limits хотя бы для одного ресурса
 - Best Effort: не установлены ограничения

Работа продемонстрировала важность корректной настройки ограничений ресурсов для обеспечения стабильности кластера и справедливого распределения ресурсов между приложениями.

6 ОТВЕТЫ НА ВОПРОСЫ

1. Назовите 3 QoS-класса

Три QoS-класса в Kubernetes:

- **Guaranteed:** назначается когда для каждого контейнера в поде для memory и cpu задан request и limit, причем эти значения совпадают. Такие поды имеют наивысший приоритет и вытесняются последними.
- **Burstable:** назначается когда хотя бы один контейнер в поде имеет request или limit, при этом request меньше limit. Эти поды имеют средний приоритет вытеснения.
- **Best Effort:** назначается когда ни один контейнер в поде не ограничен по ресурсам. Такие поды первыми кандидаты на вытеснение при нехватке ресурсов на ноде.

2. Назовите основные ресурсы системы и единицы их измерения в Kubernetes

Основные типы ресурсов в Kubernetes:

- **CPU:** измеряется в ядрах. Можно указывать дробные значения (0.5) или в милли-CPU (500m = 0.5 CPU). 1 CPU соответствует одному виртуальному ядру на облачном провайдере или одному гипертреду на физическом процессоре.
- **Memory (RAM):** измеряется в байтах. Используются суффиксы: Ki (кибибайт), Mi (мебибайт), Gi (гибибайт). Также допустимы суффиксы степеней десяти: K, M, G.

3. Для чего нужен HPA?

HPA (Horizontal Pod Autoscaler) используется для автоматического масштабирования количества подов в зависимости от использования ресурсов.

HPA работает по следующему алгоритму:

- Определяет текущие показания наблюдаемого ресурса (currentMetricValue)

- Определяет желаемые значения для ресурса (desiredMetricValue), которые задаются через request
 - Рассчитывает желаемое количество реплик по формуле: $\text{desiredReplicas} = \text{currentReplicas} * (\text{currentMetricValue} / \text{desiredMetricValue})$
 - Масштабирование не происходит, если коэффициент близок к 1 (с учетом допустимой погрешности)
4. Для чего необходимо устанавливать ограничения в Kubernetes?

Установка ограничений в Kubernetes необходима по следующим причинам:

- Предотвращение потребления одним подом всех ресурсов ноды, что может привести к отказу других приложений
- Корректное планирование подов на нодах: scheduler использует requests для определения достаточности ресурсов на ноде
- Обеспечение справедливого распределения ресурсов между приложениями
- Защита от утечек памяти и runaway процессов
- Определение QoS-класса пода, который влияет на приоритет вытеснения при нехватке ресурсов

5. Что будет с узлом при превышении ограничений?

При превышении ограничений происходит следующее:

- При превышении memory limit: контейнер немедленно убивается kubelet с ошибкой OOMKilled (Out Of Memory). После этого контейнер может быть перезапущен в соответствии с политикой перезапуска пода.
- При превышении CPU limit: процесс ограничивается через механизм throttling (троттлинг). Контейнер не завершается, но его процессорное время ограничивается установленным лимитом.
- При нехватке ресурсов на ноде: kubelet начинает ранжировать и вытеснять поды по определенному алгоритму, учитывающему приоритет пода и его QoS-класс. При одинаковом приоритете в первую очередь вытесняются

поды с классом Best Effort, затем Burstable, и в последнюю очередь Guaranteed.

7 СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Kubernetes: лучшие практики. — СПб.: Питер, 2021. — 288 с.: ил. — (Серия «Для профессионалов»).
2. K8S для начинающих. Первая часть — Текст: электронный [сайт]. — URL: <https://habr.com/ru/post/589415/> (дата обращения: 23.11.2025).
3. Kubernetes или с чего начать, чтобы понять что это и зачем он нужен — Текст: электронный [сайт]. — URL: <https://habr.com/ru/company/otus/blog/537162/> (дата обращения: 23.11.2025).
4. Основы Kubernetes — Текст: электронный [сайт]. — URL: <https://habr.com/ru/post/258443/> (дата обращения: 23.11.2025).